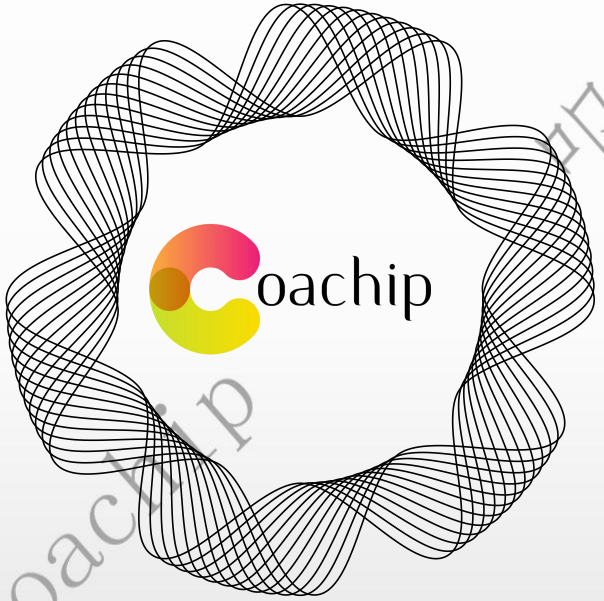


SV的随机约束

叩特电子Coachip

课程内容及目标

课程内容：

- 为什么引入随机化
- 什么需要随机化
- SV的随机化
- 约束的种类



叩特电子Coachip

为什么引入随机化(WHY)

- 芯片体积增大，复杂度日渐提高，定向测试已经无法满足验证的需求，而随机测试的比例逐渐提高。
- 定向测试能找到你认为可能存在的缺陷，而随机测试可以找到连你都没有想到的缺陷。
- 随机测试的环境要求比定向测试复杂，它需要激励、参考模型和在线比较。上百次的仿真不再需要人为参与，以此来提高验证效率。
- 随机测试相对于定向测试可以减少相当多的代码量，而产生的激励较定向测试也更多样。

叩特电子Coachip

什么需要随机化(WHAT)

- 器件配置：通过寄存器和系统信号
- 环境配置：随机化验证环境，例如合理的时钟和外部反馈信号
- 原始输入数据：例如数据包的长度、带宽，数据间的顺序
- 延时：握手信号之间的时序关系，例如valid和ready，req和ack之间的时序关系
- 协议异常：如果反馈信号给出异常，那么设计是否可以保持后续数据处理的稳定性呢？

叩特电子Coachip

System Verilog中的随机化

- 带有随机变量的简单类

```
class Packet;
  // The random variables
  rand  bit [31:0] src, dst, data[8];
  randc bit [7:0] kind;
  // Limit the values for src
  constraint c
  {
    src > 10;
    src < 15;
  }
endclass
```

```
program testgen;
  Packet p;

  initial begin
    p = new();// Create a packet
    assert(p.randomize)
    else $fatal(0, "Packet::randomize failed");
  end
endprogram
```

叩特电子Coachip

随机化约束的种类

- 布尔表达式 Boolean expressions
- 权重分配 Weighted distributions
- 范围表达式 Range expressions
- 条件表达式 Conditional expressions
- 外部约束
- 双向约束

叩特电子Coachip

System Verilog约束的种类

- 布尔表达式 Boolean expressions

```
class order;
    rand bit [7:0] lo, med, hi;
    constraint bad
    {
        lo < med < hi;
    }
endclass
```

```
class order;
    rand bit [7:0] lo, med, hi;
    constraint right
    {
        lo  < med;  // right
        med < hi;   // right
    }
endclass
```

叩特电子Coachip

System Verilog约束的种类

- 权重分配 Weighted Distributions

```
rand int src, dst;
    constraint c_dist
    {
        src dist {0:=40, [1:3]:=60};
        // src = 0, weight = 40/220
        // src = 1, weight = 60/220
        // src = 2, weight = 60/220
        // src = 3, weight = 60/220

        dst dist {0:/40, [1:3]:/60};
        // dst = 0, weight = 40/100
        // dst = 1, weight = 20/100
        // dst = 2, weight = 20/100
        // dst = 3, weight = 20/100
    }
```

叩特电子Coachip

System Verilog约束的种类

- 带变量的权重分配 Weighted Distributions with Veriable

```
typedef enum {READ8, READ16, READ32} read_e;
class ReadCommands;
    rand read_e read_cmd;
    int read8_wt=1, read16_wt=1, read32_wt=1;
    constraint c_read
    {
        read_cmd dist
        {
          READ8  := read8_wt,
           READ16 := read16_wt,
           READ32 := read32_wt
        };
    }
endclass
```

叩特电子Coachip

System Verilog约束的种类

- 范围表达式 Range expressions
  - **规定随机值的上下限**

```
rand int c; // Random variable
int lo, hi; // Non-random variables
constraint c_range
{
    c inside {[lo:hi]}; // lo <= c && c <= hi
}
```

  - **随机值的上下限取反**

```
rand int c; // Random variable
int lo, hi; // Non-random variables
constraint c_range
{
    !(c inside {[lo:hi]}); // c < lo or c > hi
}
```

叩特电子Coachip

System Verilog约束的种类

- $指定上下限

```
rand bit [6:0] b; // 0 <= b <= 127
rand bit [5:0] e;
constraint c_range
{
    b inside {[$:4], [20:$]}; // 0 <= b <= 4 || 20 <= b <= 127
    e inside {[$:4], [20:$]}; // 0 <= e <= 4 || 20 <= e <= ?
}
```

- 使用数组

```
rand bit [6:0] b; // 0 <= b <= 127
int fib [5] = '{1, 5, 3, 9, 2};
constraint c_range
{
    b inside fib;
}
```

叩特电子Coachip

System Verilog约束的种类

- 条件表达式 Conditional expressions

```
class Stim;
    bit flag;
    rand bit [31:0] dst;

    constraint c_stim
    {
      if (flag)
      {// not begin
        dst inside {[40:80]};
      }// not end
      else
        dst inside {[2:10], [50:67]};
    }
endclass
```

```
class Stim;
  bit flag;
  rand bit [31:0] dst;

  constraint c_stim
  {
    flag -> dst inside {[40:80]};
    !flag -> dst inside {[2:10], [50:67]};
  }
endclass
```

叩特电子Coachip

System Verilog约束的种类

- 外部约束

```
class Packet;
  rand bit [7:0] length;
  rand bit [7:0] payload[];
  constraint c_valid {length > 0;payload.size() == length;}
  constraint c_external;
endclass
constraint Packet::c_external {length == 1;}      //在类外定义
```

叩特电子Coachip

System Verilog约束的种类

➢ 双向约束

约束块不像自上而下执行的程序性代码，它们是声明性的代码，是并行执行的。所有的约束表达式同时有效。故SV会同时计算所有的随机变量约束，取它们的交集。

```
rand logic [15:0] r,s,t;

constraint c_bitir {
    r < t;
    s ==r;
    t <30;
    s >25;
}
```

解	r	s	t
A	26	26	27
B	26	26	28
C	26	26	29
D	27	27	28
E	27	27	29
F	28	28	29

叩特电子Coachip

练习

```
class Packet;
  // The random variables
  rand  bit [31:0] src, dst, data[8];
  randc bit [2:0] kind;
  // Limit the values for src
  constraint c
  {
    src > 10;
    src < 15;
  }
endclass
```

```
program testgen;
  Packet p;

  initial begin
    p = new();// Create a packet
    assert (p.randomize)
    else $fatal(0, "Packet::randomize failed");
    $display("src= %d",p.src);
    $display("dst= %d",p.dst);
    $display("data= %d",p.data);
    $display("kind= %d",p.kind);
  end
endprogram
```

改变constraint来改变随机化的值，
并观察打印结果是否和预想constraint一致

叩特电子Coachip

Q&A

昂特电子Coachip